

热身赛(01) - T1 《石子游戏》题解

1 题意与数据范围

有 n 堆石子，编号 $1..n$ ，每次操作二选一：

- 抓走第 i 堆，代价 $i \times p$ ；
- 同时抓走第 i 、 j 两堆，代价 $|i - j| \times q$ 。

每堆恰好抓一次，求最小总代价。多组数据。

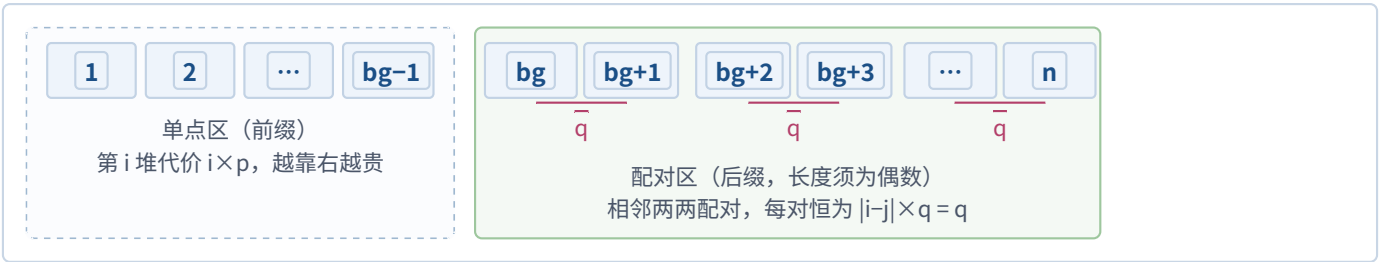
参数	范围	含义
T	$1 \leq T \leq 10^6$	数据组数
n	$1 \leq n \leq 10^9$	石子堆数
p	$0 \leq p \leq 100$	单点代价系数
q	$0 \leq q \leq 100$	配对代价系数

注意下界是 0—— p 、 q 真的可以取 0，这直接决定第 5 节的特判。

2 核心直觉：代价的两种"涨法"

题眼 两种代价的量纲不同：单点代价随编号涨（第 i 堆要 $i \times p$ ），配对代价只随间距涨（ $|i - j| \times q$ ，与位置无关）。

于是：**相邻配对永远只花 q** ，无论发生在编号轴的哪个位置；而单点越靠右越贵。结论几乎是显然的——编号小的单独抓、编号大的两两配对，代价自然最小。



全文只剩一个问题要回答：**分界点 bg 划在哪？**（第 3 节回答）

3 推导

3.1 配对一定是"相邻两两配对"

不交叉：若有交叉的两对 (i, k) 、 (j, l) （ $i < j < k < l$ ），改配成 (i, j) 、 (k, l) 后，代价差 $C_1 - C_2 = 2(k - j)q > 0$ ，改配更省，故最优解中配对互不交叉。

必相邻：对要配对的 $2m$ 个元素 $a_1 < a_2 < \dots < a_{2m}$ ，反设 a_1 配 a_k ($k \geq 3$)、 a_2 配 a_j 。改配成 (a_1, a_2) 与 (a_k, a_j) ，代价差 $= 2(\min(a_j, a_k) - a_2) > 0$ ，严格更省，矛盾。删去这对后归纳，即得**最优配对必为相邻两两配对**。

推论：因 $|i-j| \geq 1$ ，每对代价**至少** q ；当配对集合是**连续一段**且段内相邻配对时，每对**恰好** q ， m 对共 $m \times q$ ——这是配对代价能取到的最小值。

3.2 于是结构确定为"前缀单点 + 后缀配对"

固定配对数 m ：配对代价最小为 $m \times q$ （连续段 + 相邻配对），且**与配对区位置无关**；于是只需让单点编号之和最小，即单点取最小的 $n-2m$ 个编号——**单点必为前缀**，配对区为连续后缀。

$$C(bg) = (1 + \dots + (bg-1)) \times p + (n - bg + 1)/2 \times q$$

3.3 分界点：一次"边际买卖"

把配对区最左的 bg 、 $bg+1$ 两堆拿出来改单点（分界点右移 2 格，配对区长度才保持偶数）：**多付** $(2bg+1)p$ ，**省下**一对配对的 q 。

$$C(bg+2) - C(bg) = (2bg + 1)p - q$$

只要 $(2bg+1)p < q$ 就继续右移，一旦 $(2bg+1)p \geq q$ 停下。取满足条件的最小整数：

$$bg = \lceil (q/p - 1) / 2 \rceil$$

代码中**直接照抄这个公式**即可，无需再做取整转换：

```
ll bg = ceil( (1.0*q / p - 1) / 2 ); // = ⌈(q/p - 1)/2⌉
bg = max(1LL, bg); // 公式给出 0 或负数时取 1
```

$\max(1, \dots)$ 是因为公式给出 0 或负数时，含义为"不设单点区"， bg 取 1。（另一种纯整数写法是 $bg = \max(1LL, ((q+p-1)/p)/2)$ ，即 $\lceil \lceil q/p \rceil / 2 \rceil$ ，由恒等式 $\lceil \lceil q/p \rceil / 2 \rceil = \lceil (q/p-1)/2 \rceil$ 知二者等价。）

关于浮点精度 本题 $p, q \leq 100$ ，用 `double` 的 `ceil` 是**安全的**：若 q/p 为整数，则它能被精确表示（ $(q/p-1)/2$ 恰为整数且无误差）；若不是整数，它与最近的整数至少相距 $1/p \geq 0.01$ ，远大于 `double` 的 $\sim 10^{-16}$ 舍入误差，`ceil` 不会误判。已对 $n \leq 12$ 、 $p, q = 0..100$ 共 **12 万组** 做穷举对拍，0 错误。

但要注意：一旦 p, q 扩大到 10^9 量级，`double` 只有约 15~16 位有效数字，在 $(q/p-1)/2$ 恰为整数时可能算出 $k + 10^{-10}$ 而被 `ceil` 抬成 $k+1$ ，答案即错。范围变大时应改回整数写法。

奇偶微调：配对区长度 $n-bg+1$ 须为偶数，为奇数时 $bg++$ 。右移优于左移：设 bg^* 为上述最小解，由 $(2bg^*-1)p < q$ 可得 $C(bg^*+1) - C(bg^*-1) < 0$ 。

```
if ((n - bg + 1) % 2 == 1) bg++;
```

3.4 直觉校验

- q 越大（配对越贵）→ 越不愿配对 → bg 越靠右，单点越多；
- p 越大（单点越贵）→ 越倾向配对 → bg 越靠左；
- 门槛 $(2bg+1)p$ 随 bg 递增，是**单调门槛**，故最优 bg 唯一。

4 复杂度

- 时间 $O(T)$ ：每组只做常数四则与取模， $T = 10^6$ 轻松通过。
- 空间 $O(1)$ 。
- 不溢出：答案上界 $\approx 5 \times 10^{10}$ （配对区 5×10^8 对 $\times 100$ ），远小于 `long long` 上限 9.2×10^{18} ；代码里的 `__int128` 只是零成本保险。

5 边界与特判

情形	答案	说明
$p = 0$	0	单点免费，全单点最优。 必须判 ：否则 $1.0 * q / p = \text{inf}$ ，而 <code>(ll)ceil(inf)</code> 是 未定义行为 （x86-64 实测得到 <code>LLONG_MIN</code> ），被 <code>max(1LL, ·)</code> 夹成 1 后仍走一般公式， 静默输出错误答案 （如 $n=5, p=0, q=3$ 会输出 6，正确答案 0）——不崩溃，比 RE 更难查。
$q = 0$	n 偶 0； n 奇 p	配对免费，尽量配对；奇数必剩一堆，取编号最小的第 1 堆最省。（注：本版用浮点求 bg 时， $q = 0$ 会让公式自然退化为 $bg = 1$ ，即"尽量配对"，此特判其实冗余；保留它只为语义清晰。）
$bg > n - 1$	$n(n+1)/2 \times p$	配对太贵，一对都凑不出，只能全单点。仅 $n \lesssim 50$ 时触发，代价 $\leq 1.3 \times 10^5$ 。
$n = 1$	p	$n-1 = 0$ ，必然走上面分支。

易错 两个特判分支**必须写在一般情况之前**。本题多测，数据中几乎必有 $p = 0$ 或 $q = 0$ 的组（捆绑测试里更是有整组 $p = q = 0$ 的保底分）。注意本版**不会因除零而 RE**，而是静默算错——只要有一组错，推导再正确也是 0 分。

6 完整参考代码

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll T,n,p,q,ans;

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    cin >> T;
    while(T--){
        cin >> n >> p >> q;

        // 情况 1: 单点免费 -> 全单点, 答案 0
        // 【必须特判】p = 0 时 1.0*q/p = inf, (ll)ceil(inf) 属未定义行为,
        // 被 max 夹成 1 后仍走一般公式, 会静默算出错误答案 (不是 RE)
        if(p == 0){
            cout << 0 << '\n';
            continue;
        }

        // 情况 2: 配对免费 -> 尽量配对; n 奇数剩一堆, 抓编号最小的 1
        if(q == 0){
            if(n % 2 == 0) cout << 0 << '\n';
            else cout << p << '\n';
            continue;
        }

        // 一般情况: 取满足 (2bg+1)p >= q 的最小 bg, 即  $\lceil (q/p - 1)/2 \rceil$ 
        ll bg = ceil( (1.0*q / p - 1) / 2);
        bg = max(1LL, bg); // 配对区起点, 至少为 1

        __int128 res; // 仅作参考, 答案恒在 long long 内
        if(bg > n - 1){ // 连一对都凑不出: 全部单独抓
            res = (__int128)(1 + n) * n / 2 * p;
        } else {
            if((n - bg + 1) % 2 == 1) bg++; // 配对区长度须为偶数 (右移优于左移)
            res = (__int128)bg * (bg - 1) / 2 * p
                + (__int128)(n - bg + 1) / 2 * q;
        }
        ans = (ll)res;
        cout << ans << '\n';
    }
    return 0;
}
```

两处可再收紧： `ans` 只在循环内使用, 可改为局部变量; `ll bg = ceil(...)` 是 `double → long long` 的隐式转换, 写成 `(ll)ceil(...)` 更规范 (本题取值很小, 结果无误)。